R.real.10

Complément Web

React Avancé

Limitations

- Avec babel Stand Alone il est difficile d’utiliser plusieurs modules
- Nous avons plusieurs pages, il est difficile de partager des ressources
- Nous ne pouvons maintenir un état partagé entre deux hiérarchies de composants

Environnement de développement

Un gestionnaire de paquets

Un bundler

Un compilateur

⇒create-react-app

Rem : esLint

npx create-react-app monapp

npm run start

npm run test

npm run build

npm run eject

Single Page Application

Une single-page application (SPA) est une application qui charge une unique page HTML et toutes les ressources nécessaires (telles que du JavaScript et des CSS) requises pour le fonctionnement de l'application.

Le but est d’éviter le chargement d’une nouvelle page et de factoriser les ressources.

Routage

Le routage permet de modifier une partie de la page en fonction de l'URL.

En React la module par défaut est React Router : https://reactrouter.com/en/main

Routage composants

- Installation

  npx create-react-app react_spa
  cd react_spa
  npm i react-router-dom

- Des composants fournis
  - Routers
    - <BrowserRouter> utilise History API de l'HTML 5
    - <HashRouter> utilise le hash de l'URL (window.location.hash)
    - <MemoryRouter> travail en mémoire pas d'historique
    - <NativeRouter> pour React Native
    - <Router> Un context provider (bas niveau)
    - <StaticRouter> Pour node

Routage composants

- <Route/> Association entre une URL et un composant

- Composants
  - <Link>
  - <NavLink>
  - …

Routage exemple

- index.html

```
<div id="root"></div>
```

- index.js

```
<React.StrictMode>
  <Main/>
</React.StrictMode>
```

Routage exemple

- main.js

```
import {
    Routes,
    Route,
    NavLink,
    HashRouter
} from "react-router-dom";
```

Avec But2,S3,S4 des composants

```
<HashRouter>
<div>
   <h1>S4</h1>
   <ul className="header">
      <li><NavLink to="/">BUT2</NavLink></li>
      <li><NavLink to="/S3">S3</NavLink></li>
      <li><NavLink to="/S4">S4</NavLink></li>
   </ul>
   <div className="content">
      <Routes>
      <Route exact path="/" Component={But2}
/>
      <Route path="/S3" Component={S3} />
      <Route path="/S4" Component={S4} />
      </Routes>
   </div>
</div>
</HashRouter>
```

Etat global

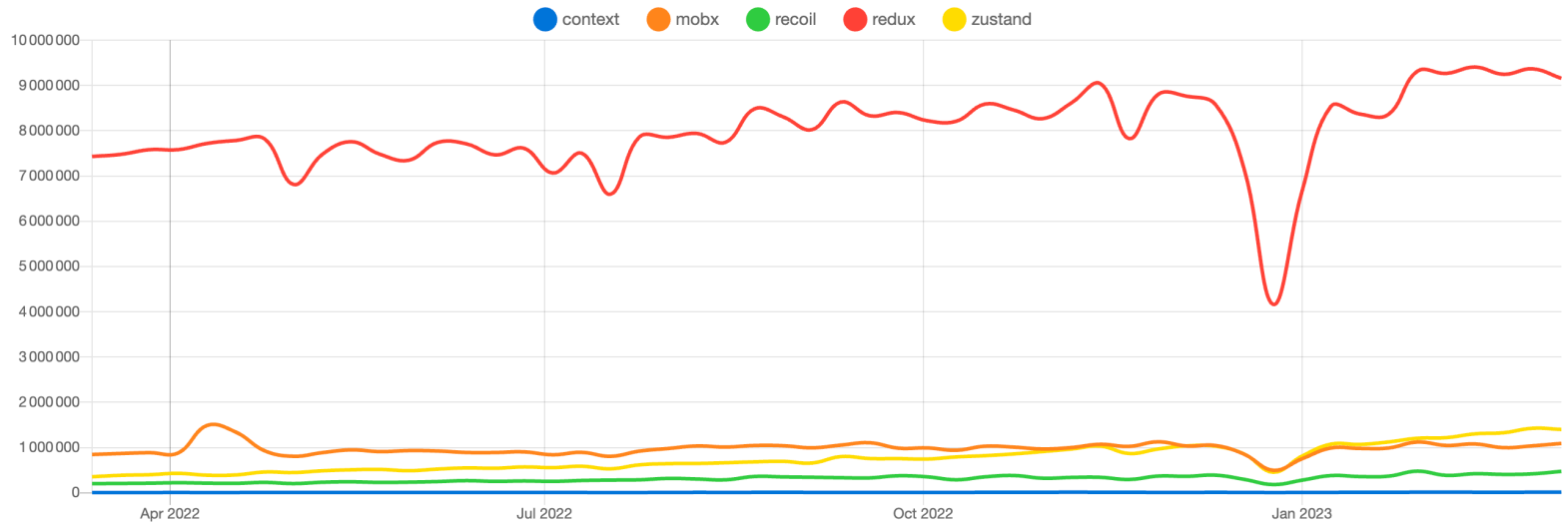
https://npmtrends.com/context-vs-mobx-vs-recoil-vs-redux-vs-zustand

context vs **mobx** vs **recoil** vs **redux** vs **zustand**

Enter an npm package...

context × mobx × recoil × redux × zustand × + react-redux + jotai

Downloads in past 1 Year



Stats

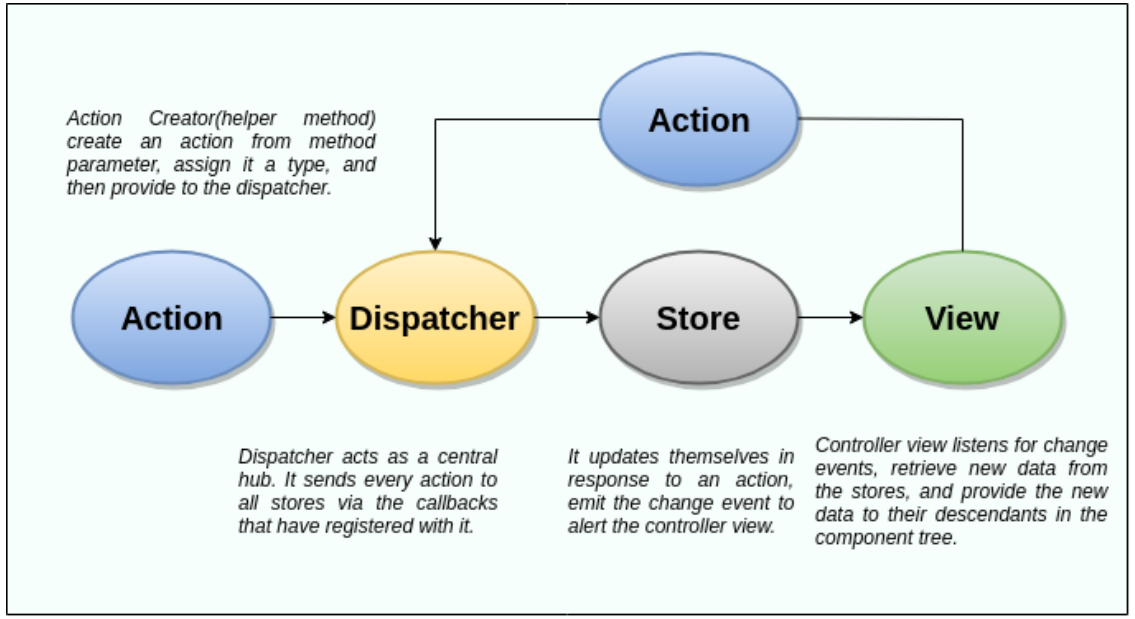
Redux

https://redux.js.org/

Basé sur le design pattern flux

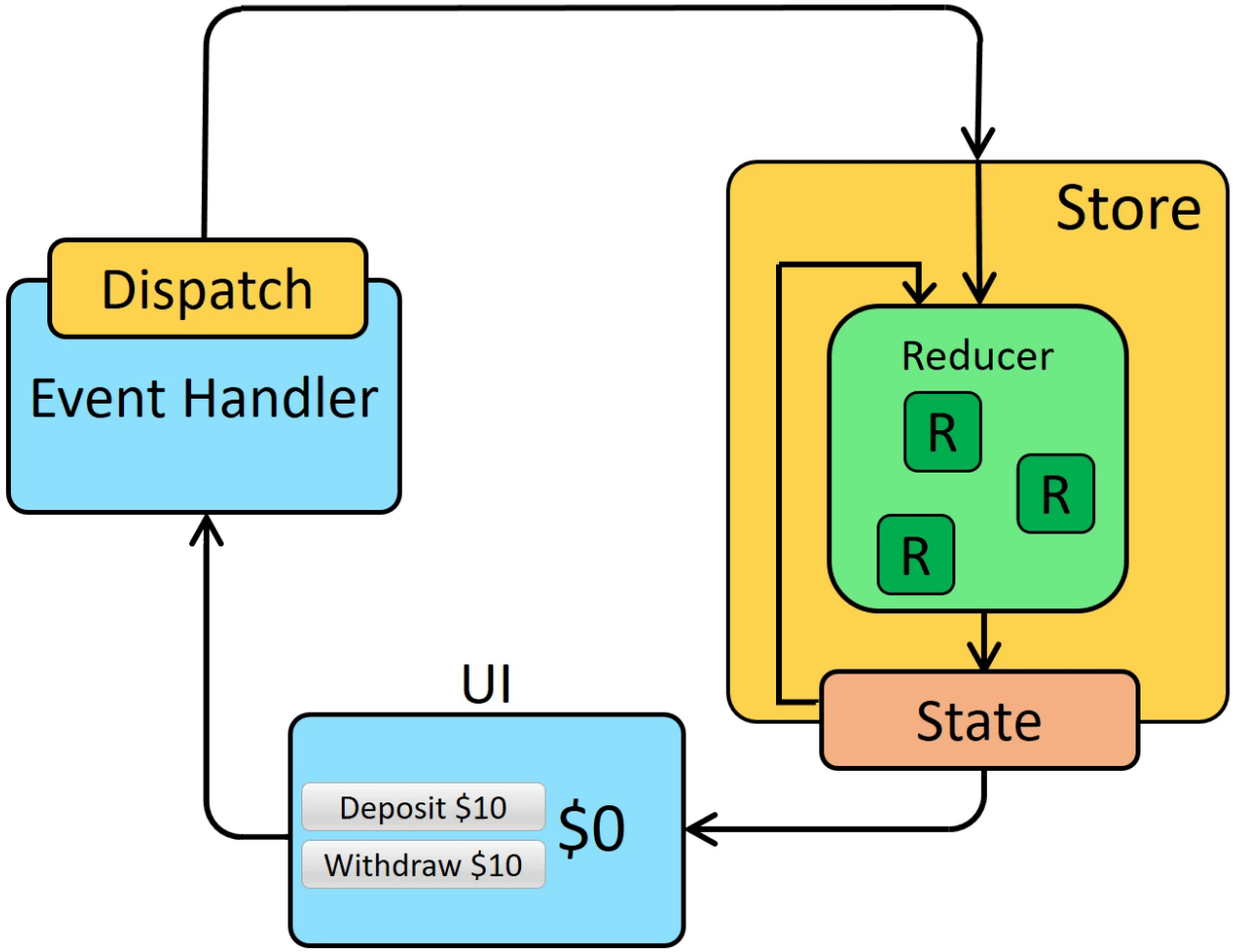
Des concepts :

- State état global modifié par des actions
- Une action un objet avec type et payload
- Reducer (state,action) -> state
- Store (



https://www.javatpoint.com/react-flux-concept

Redux


Dispatch
Event Handler
Store
Reducer
R
R
R
State
UI
Deposit $10
Withdraw $10
$0

Exemple

Un provider doit englober l'application et définir le store :

```
<Provider store={store}>
  <StrictMode>
    <App />
  </StrictMode>
</Provider>
```

//App.js

```
<div>
  <Display />
  <div className="buttons-row">
    <PointScoredButton playerId="player1">Point Joueur
1</PointScoredButton>
    <PointScoredButton playerId="player2">Point Joueur
2</PointScoredButton>
  </div>
  <div className="buttons-row">
    <ResetButton />
    <PlayPauseButton />
  </div>
</div>
```

https://openclassrooms.com/fr/courses/7150626-utilisez-le-state-manager-redux-pour-gerer-l-etat-de-vos-applications

Action

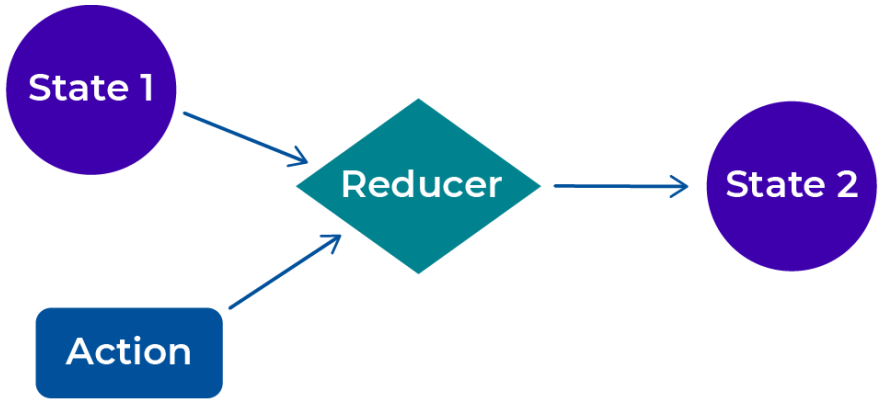
Une action : Un objet avec un type et éventuellement un payload

Action Creator : une fonction qui crée des actions

```
export const pointScored = (player) => ({
  type: "pointScored",
  payload: { player: player },
});
```

Reducer

Le passage d’un état à un autre en fonction des actions



Reducer

```
function reducer(state = initialState,
action) {
  if (action.type === "restart") {
    return initialState;
  }
  if (action.type === "playPause") {
    if (state.winner) {
      return state;
    }
    return {
      ...state,
      playing: !state.playing,
    };
  }
```

```
const initialState = {
  player1: 0,
  player2: 0,
  advantage: null,
  winner: null,
  playing: true,
};
```

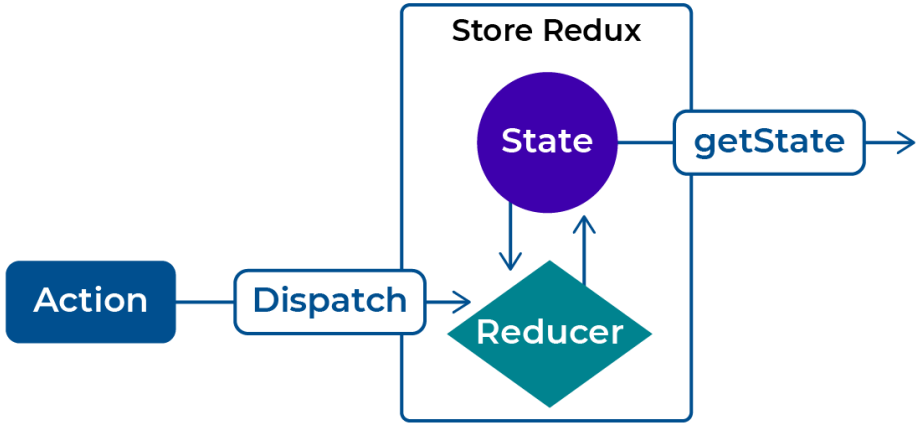
Rem : … pour une création pas d'alias

Le store

- Créé avec ***createStore***(reducer, [initial State]);

```
// on crée le store avec le state et le reducer
const store = createStore(reducer, initialState);
const state = store.getState();
```

Redux : dispatch



Manipulations :

dispatch remplace setState

store.dispatch(action); //action obtenue avec une fonction créatrice

Redux : subscribe

Pour être notifié d’un changement d’état

```
store.subscribe(
    // cette fonction sera exécutée à chaque fois que le state change
    () => {
store.getState();

updateScoreText(state.playing);
    }
);
```

Redux : Hooks (envoyer des actions)

- Custom hooks :

1. Import : `import { useDispatch } from "react-redux";`
2. Utilisation

```
const dispatch = useDispatch();
dispatch(action);
```

Redux : State dans les composants

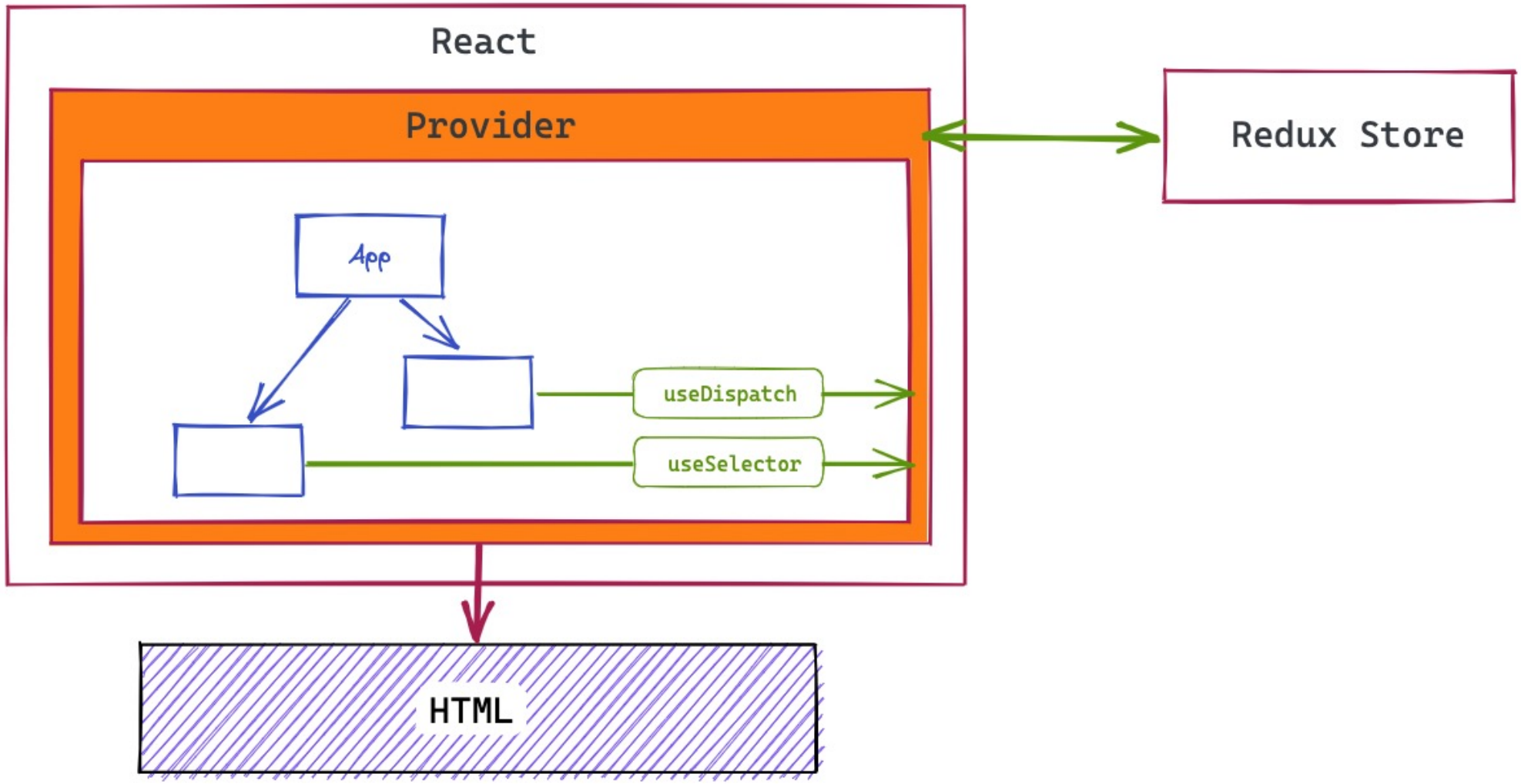
```
import { useSelector } from "react-
re
```

```
const gameIsPlaying =
useSelector((state) =>
state.playing);
```

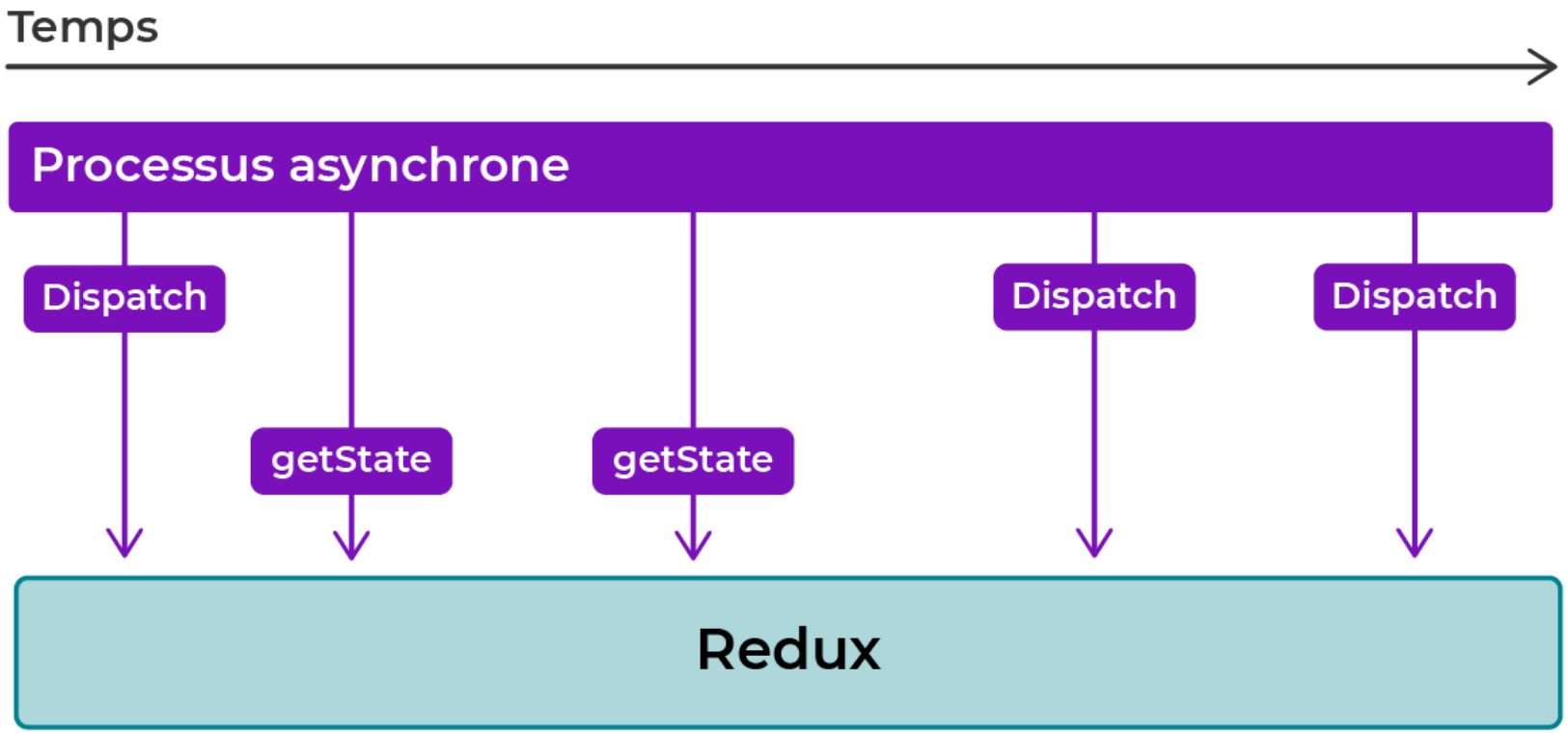
Mise à jour automatique du composant lors du changement de State en plus de la lecture

useSelector utilise le `store.subscribe()`

React : Synthèse



Redux : Asynchronisme



Redux : Asynchrone

- Redux **ne peut pas manipuler du code asynchrone** directement, en revanche on peut interagir avec Redux depuis une fonction asynchrone

- Récupérer le store dans un composant
  - import { useStore } from "react-redux";
  - const store = useStore();

  Puis useState et dispatch

Redux

Limites :

- faible pour les manipulations asynchrones
- Grande quantité de code

Solutions :

- Redux-Saga pour l’asynchronisme (complexe)
- React-Query

React Native

- npx create-expo-app Demo
- npx expo install react-dom react-native-web @expo/webpack-config
- npx expo start